

STRINGS

CHECK SAME

BINARY NUMBER

int x=20;
10100

Given two characters in the input, check whether they are same or not.

a b

a a

NO

YES

```
char ch1, ch2;
```

```
cin >> ch1 >> ch2;
```

```
if (ch1 == ch2)
```

```
{  
    cout << "Yes";
```

```
}
```

```
else
```

```
{  
    cout << "No";
```

```
}
```

ch1 ch2

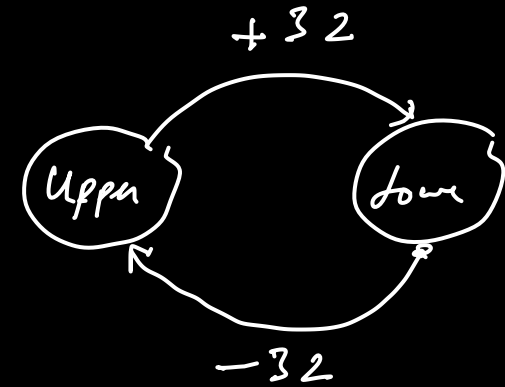
ASCII VALUES

Every basic character on the keyboard is assigned a unique numeric value by the computer, called its ASCII value.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]



Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
32	20	[SPACE]	64	40	@	96	60	`
33	21	!	65	41	A	97	61	a
34	22	"	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	\$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	'	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2A	*	74	4A	J	106	6A	j
43	2B	+	75	4B	K	107	6B	k
44	2C	,	76	4C	L	108	6C	l
45	2D	-	77	4D	M	109	6D	m
46	2E	.	78	4E	N	110	6E	n
47	2F	/	79	4F	O	111	6F	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3A	:	90	5A	Z	122	7A	z
59	3B	;	91	5B	[	123	7B	{
60	3C	<	92	5C	\	124	7C	
61	3D	=	93	5D	]	125	7D	}
62	3E	>	94	5E	^	126	7E	~
63	3F	?	95	5F	_	127	7F	[DEL]



char ch;

cin >> ch;

if (ch >= 'a' and ch <= 'z')

{
 cout << "lower";
}

else if (ch >= 'A' and ch <= 'Z')
{
 cout << "upper";
}

CHARACTER TO ASCII

Given a character, print its ASCII Value

A

a

65

97

```
        cout << "upper";  
    }  
    ch if(ch >= '0' and ch <= '9')  
    {  
        cout << "digit";  
    }  
    char ch; }  
    cin >> ch; }  
    {  
        cout << "Special";  
    }  
    }  
    cout << (int) ch;
```

ASCII TO CHARACTER

Given an ASCII Value, print the corresponding character

65

97

```
int val;  
cin >> val;  
  
cout << (char) val;
```

A

a

CONVERT TO LOWERCASE

Given an alphabet (lowercase/uppercase), convert into lowercase.



```
char ch;
```

```
cin >> ch;
```

```
if( ch >= 'A' and ch <= 'Z' )
```

```
{
```

```
    ch += 32;
```

```
}
```

```
cout << ch;
```

CHECK CHARACTER

Check whether a given character is lowercase, uppercase , digit or a special character.

a

B

7

@

lower

upper

digit

special

"Dhruv"



char s[5];

STRING

- **Collection / Sequence of characters** stored at continuous memory locations.
- In C++, there are 2 ways to implement it:

1) Char Array (C style)

- `char s[100];`
- Ends with `'\0'`
- Manual memory handling
- More error-prone

2) String (STL)

- `string s;`
- Easier
- Automatic memory management
- Less error-prone
- More inbuilt functions available

